UNIVERSIDAD POLITÈCNICA DE ATLAUTLA

MATERIA: FUNDAMENTOS DE INTELIGENCIA ARTIFICIAL

SECCIÒN: 2

2.1 Problemas de bùsqueda

2.2 Espacios de bùsqueda

DOCENTE: JAVIER GARCÌA LARA

ALUMNA: ALEXANDRA BERNAL NÀJERA

GRUPO: 171

Gemini (Google DeepMind) / Modelo de Lenguaje de Gran Escala (LLM).

Planteamiento del Problema

PROBLEMA 1: OPTIMIZACIÓN DE RUTA DE ENTREGA LOGÍSTICA (ÚLTIMA MILLA)

Una empresa de logística requiere optimizar las entregas de paquetes utilizando un vehículo que parte de un almacén central. El vehículo debe visitar a una serie de clientes para entregar paquetes pendientes y regresar al almacén minimizando la distancia total recorrida.

Datos del Problema

- Nodos principales: Almacén central, Cliente A, Cliente B.
- Matriz de Distancias (Red Vial Ponderada):
    - Almacén → Cliente A: 5 km
    - Almacén → Cliente B: 10 km
    - Cliente A → Cliente B: 4 km
    - Cliente B → Cliente A: 4 km
    - Cliente A → Almacén: 5 km
    - Cliente B → Almacén: 8 km

Formulación Formal del Problema

1. Estado (S): Tupla (ubicacion_actual, paquetes_pendientes) donde ubicacion_actual es una cadena con el nombre de la ubicación y paquetes_pendientes es una tupla con los clientes que aún faltan por recibir su paquete.
2. Estado Inicial (S_0): ('Almacen', ('Cliente_A', 'Cliente_B'))

3. Acciones ($A(s)$):
    - Moverse(origen, destino): Desplazar el vehículo a un nodo adyacente en el grafo.
    - Entregar(cliente): Entregar el paquete si el vehículo se encuentra en la ubicación del cliente pendiente.
4. Test de Meta ($TestMeta(s)$): Evalúa si el estado actual es igual a ('Almacen', ()) (Regresar al almacén sin paquetes pendientes).

5. Costo de Paso ($C(s, a, s')$): Distancia en kilómetros de la arista transitada. El acto de entrega en el lugar tiene costo 0.

Código en Python (Modelo + Generador de Estados + NetworkX)

```
import networkx as nx

import matplotlib.pyplot as plt


# 1. CONSTRUCCIÓN DEL GRAFO DIRIGIDO PONDERADO

G_logistica = nx.DiGraph()


conexiones_logistica = [

    ('Almacen', 'Cliente_A', 5),

    ('Almacen', 'Cliente_B', 10),

    ('Cliente_A', 'Cliente_B', 4),

    ('Cliente_B', 'Cliente_A', 4),

    ('Cliente_A', 'Almacen', 5),

    ('Cliente_B', 'Almacen', 8)

]


for origen, destino, costo in conexiones_logistica:

    G_logistica.add_edge(origen, destino, weight=costo)


# 2. GENERADOR DE ESTADOS SUCESORES
```

```
def generador_estados_logistica(estado_actual, grafo):
    """
    Recibe el estado actual (ubicacion, pendientes) y genera
    todos los estados alcanzables mediante acciones válidas.
    """
    ubicacion, pendientes = estado_actual
    sucesores = []

    # Acción A: Entregar paquete
    if ubicacion in pendientes:
        nuevos_pendientes = tuple(p for p in pendientes if p != ubicacion)
        sucesores.append((
            (ubicacion, nuevos_pendientes),
            0,
            f"Entregar paquete en {ubicacion}"
        ))

    # Acción B: Desplazarse a un nodo adyacente
    for vecino in grafo.successors(ubicacion):
        costo = grafo[ubicacion][vecino]['weight']
        sucesores.append((
```

```
            (vecino, pendientes),
            costo,
            f"Moverse a {vecino}"
        ))

    return sucesores

# 3. GENERACIÓN DEL DIAGRAMA DEL GRAFO
plt.figure(figsize=(7, 5))
pos = nx.spring_layout(G_logistica, seed=42)
nx.draw_networkx_nodes(G_logistica, pos, node_size=2000, node_color='skyblue', edgecolors='black')
nx.draw_networkx_labels(G_logistica, pos, font_size=10, font_weight='bold')
nx.draw_networkx_edges(G_logistica, pos, arrowstyle='->', arrowsize=20, edge_color='gray', width=2)

labels = nx.get_edge_attributes(G_logistica, 'weight')
labels_formatted = {k: f"{v} km" for k, v in labels.items()}
nx.draw_networkx_edge_labels(G_logistica, pos, edge_labels=labels_formatted)

plt.title("Grafo Dirigido Ponderado - Ruta de Entrega Logística", fontsize=11, fontweight='bold')
plt.axis('off')
```

```
plt.tight_layout()

plt.savefig("diagrama_grafo_problema1.png", dpi=300)

plt.show()
```

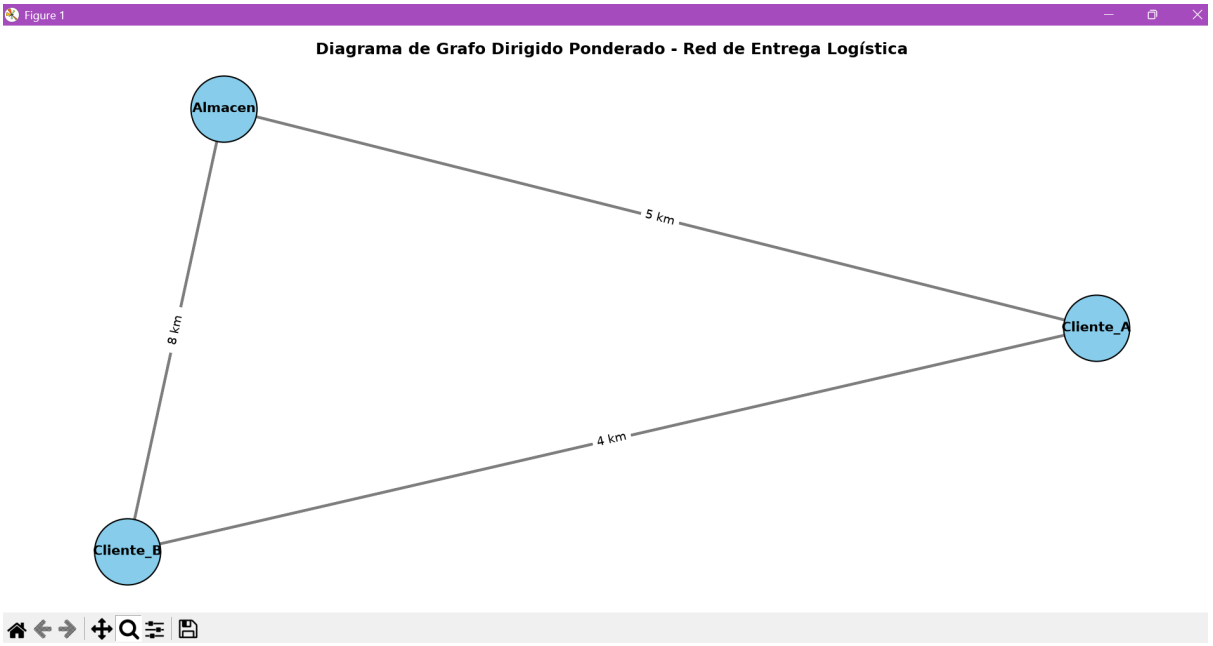
Procedimiento y Solución

Ejemplo de traza del espacio de estados partiendo de S_0:

1. Estado Inicial: ('Almacen', ('Cliente_A', 'Cliente_B'))
2. Sucesores desde S_0:
    - Moverse a Cliente_A → Nuevo Estado: ('Cliente_A', ('Cliente_A', 'Cliente_B')), Costo acumulado: 5 km.
    - Moverse a Cliente_B → Nuevo Estado: ('Cliente_B', ('Cliente_A', 'Cliente_B')), Costo acumulado: 10 km.
3. Desde **('Cliente_A', ('Cliente_A', 'Cliente_B'))**:
    - Entregar en Cliente_A → Nuevo Estado: ('Cliente_A', ('Cliente_B',)), Costo acumulado: 5 km.
4. Desde **('Cliente_A', ('Cliente_B',))**:
    - Moverse a Cliente_B → Nuevo Estado: ('Cliente_B', ('Cliente_B',)), Costo acumulado: 9 km.
5. Desde **('Cliente_B', ('Cliente_B',))**:
    - Entregar en Cliente_B → Nuevo Estado: ('Cliente_B', ()), Costo acumulado: 9 km.
6. Desde **('Cliente_B', ())**:
    - Moverse a Almacen → Estado Meta: ('Almacen', ()), Costo total óptimo: 17 km.

Ruta Óptima Solución: Almacén → Cliente A (Entregar) → Cliente B (Entregar) → Almacén (Costo Total: 17 km).

Diagrama de Grafos



Planteamiento del Problema

PROBLEMA 2: NAVEGACIÓN DE UN ROBOT AUTÓNOMO EN PLANTA INDUSTRIAL

Un robot de transporte de material debe desplazarse en una cuadrícula de 4×4 dentro de una planta industrial, desde una estación de carga hasta una estación de ensamblado. La planta contiene obstáculos infranqueables y zonas de alto tráfico que aumentan el consumo de batería.

Datos del Problema

- Dimensiones del entorno: Cuadrícula de 4×4 (coordenadas (x, y) de $(0, 0)$ a $(3, 3)$).
- Estado Inicial: $(0, 0)$ (Estación de Carga).
- Estado Meta: $(3, 3)$ (Estación de Ensamblado).
- Obstáculos: Celdas $(1, 1)$ y $(2, 1)$ (Invisibles o no transitables en el grafo).
- Zonas de alto tráfico (Costo = 3): Celdas $(1, 2)$ y $(2, 2)$.
- Costo base por celda libre: 1 unidad de energía.

Formulación Formal del Problema

1. Estado (S): Coordenadas discretas (x, y) dentro de la cuadrícula.
2. Estado Inicial (S_0): (0, 0).
3. Acciones ($A(s)$): Mover_Norte, Mover_Sur, Mover_Este, Mover_Oeste.
4. Test de Meta ($TestMeta(s)$): Evalúa si el robot se encuentra en (3, 3).
5. Costo de Paso ($C(s, a, s')$):
   - $C = 3$ si el nodo destino está en zonas de alto tráfico.
   - $C = 1$ para desplazamientos a celdas estándar.
   - Transición no permitida (∞) hacia obstáculos.

Código en Python (Modelo + Generador de Estados + NetworkX)

```
import networkx as nx

import matplotlib.pyplot as plt


# 1. CONSTRUCCIÓN DEL GRAFO DIRIGIDO EN CUADRÍCULA

G_planta = nx.DiGraph()


filas, columnas = 4, 4

obstaculos = {(1, 1), (2, 1)}

zonas_pesadas = {(1, 2): 3, (2, 2): 3}


for x in range(filas):

    for y in range(columnas):

        if (x, y) in obstaculos:

            continue


        movimientos = [
```

```
            ('Norte', (x, y + 1)),
            ('Sur', (x, y - 1)),
            ('Este', (x + 1, y)),
            ('Oeste', (x - 1, y))
        ]

        for direccion, (nx_pos, ny_pos) in movimientos:
            if 0 <= nx_pos < filas and 0 <= ny_pos < columnas:
                if (nx_pos, ny_pos) not in obstaculos:
                    costo = zonas_pesadas.get((nx_pos, ny_pos), 1)
                    G_planta.add_edge((x, y), (nx_pos, ny_pos), weight=costo,
dir=direccion)

# 2. GENERADOR DE ESTADOS SUCESORES
def generador_estados_navegacion(posicion_actual, grafo):
    """
    Recibe la posición actual (x, y) y genera
    los movimientos válidos en la cuadrícula.
    """
    sucesores = []
    if posicion_actual in grafo:
        for vecino in grafo.successors(posicion_actual):
            costo = grafo[posicion_actual][vecino]['weight']
            direccion = grafo[posicion_actual][vecino]['dir']
            sucesores.append((
```

```
            vecino,
            costo,
            f"Mover {direccion} a {vecino}"
        ))
    return sucesores

# 3. GENERACIÓN DEL DIAGRAMA DEL GRAFO
plt.figure(figsize=(7, 7))
posicion_grid = {nodo: nodo for nodo in G_planta.nodes()}

nx.draw_networkx_nodes(G_planta, posicion_grid, node_size=1500,
node_color='lightgreen', edgecolors='darkgreen')
nx.draw_networkx_labels(G_planta, posicion_grid, font_size=8, font_weight='bold')
nx.draw_networkx_edges(G_planta, posicion_grid, arrowstyle='->', arrowsize=12,
edge_color='gray')

pesos = nx.get_edge_attributes(G_planta, 'weight')
nx.draw_networkx_edge_labels(G_planta, posicion_grid, edge_labels=pesos,
font_size=7)

plt.title("Grafo Dirigido Ponderado - Navegación de Robot en Planta", fontsize=11,
fontweight='bold')
plt.grid(True, linestyle='--', alpha=0.4)
plt.tight_layout()
plt.savefig("diagrama_grafo_problema2.png", dpi=300)
plt.show()
```

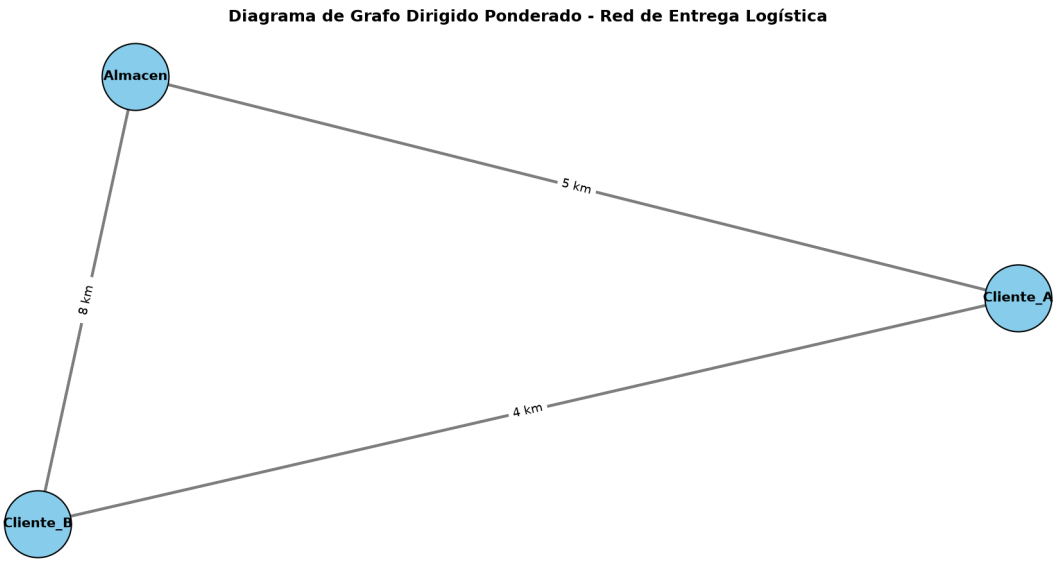
Procedimiento y Solución

Para encontrar la secuencia óptima desde $(0, 0)$ hasta $(3, 3)$:

1. Estado Inicial: $(0, 0)$
2. Nodos sucesores directos:
    - $(1, 0)$ con costo 1.
    - $(0, 1)$ con costo 1.
3. Evitación de Obstáculos: La celda $(1, 1)$ está bloqueada, obligando al agente a rodear la zona.
4. Evaluación de Zonas de Alto Tráfico:
    - Ruta A (por las celdas pesadas $(1, 2)$ y $(2, 2)$): $(0, 0) \rightarrow (0, 1) \rightarrow (0, 2) \rightarrow (1, 2)[C = 3] \rightarrow (2, 2)[C = 3] \rightarrow (3, 2) \rightarrow (3, 3)$. Costo total = $1 + 1 + 3 + 3 + 1 + 1 = 10$.
    - Ruta B (por el borde inferior/derecho rodeando obstáculos): $(0, 0) \rightarrow (1, 0) \rightarrow (2, 0) \rightarrow (3, 0) \rightarrow (3, 1) \rightarrow (3, 2) \rightarrow (3, 3)$. Costo total = $1 + 1 + 1 + 1 + 1 + 1 = 6$.

Solución Óptima: La secuencia de posiciones es $(0, 0) \rightarrow (1, 0) \rightarrow (2, 0) \rightarrow (3, 0) \rightarrow (3, 1) \rightarrow (3, 2) \rightarrow (3, 3)$ con un costo óptimo total de 6 unidades de energía.

Diagrama de Grafo



Còdigo de ambos problemas

"""

==========

SIMULACIÓN INTERACTIVA EN TIEMPO REAL - AMBOS PROBLEMAS

==========

"""

```
from collections import deque
import time
import networkx as nx

print("=======")
print(" 1. INICIANDO SIMULACIÓN EN VIVO: PROBLEMA DE LOGÍSTICA")
print("=======")
time.sleep(1)
```

Gemini (Google DeepMind) / Modelo de Lenguaje de Gran Escala (LLM).

```
# Grafo de la red vial
G1 = nx.DiGraph()
for o, d, c in [
    ('Almacen', 'Cliente_A', 5), ('Almacen', 'Cliente_B', 10),
    ('Cliente_A', 'Cliente_B', 4), ('Cliente_B', 'Cliente_A', 4),
    ('Cliente_A', 'Almacen', 5), ('Cliente_B', 'Almacen', 8)
]:
    G1.add_edge(o, d, weight=c)

def simulacion_en_vivo_logistica(estado_inicial, estado_meta):
    frontera = deque([(estado_inicial, [], 0)])
    visitados = set()
    paso = 1

    while frontera:
        actual, camino, costo_total = frontera.popleft()
        ubicacion, pendientes = actual

        print(f"\n[Paso {paso}] Analizando estado...")
        print(f"   📍 Ubicación actual del camión : {ubicacion}")
        print(f"   📦 Paquetes pendientes         : {pendientes}")
        print(f"   ⛽ Costo acumulado hasta aquí : {costo_total} km")
        time.sleep(1.5) # Pausa visual de 1.5 segundos para simular el proceso

        if actual == estado_meta:
            print("\n" + "!"*55)
            print(f" ¡ÉXITO! Vehículo de vuelta en el Almacén sin paquetes.")
            print(f" Distancia total óptima recorrida: {costo_total} km")
            print(f" Ruta óptima de acciones: {' -> '.join(camino)}")
            print("!"*55)
            return

        if actual in visitados:
            print("   -> Estado ya visitado anteriormente. Ignorando...")
```

```
            continue
        visitados.add(actual)

        # Generar sucesores de entrega
        if ubicacion in pendientes:
            nuevos_pendientes = tuple(p for p in pendientes if p != ubicacion)
            siguiente_estado = (ubicacion, nuevos_pendientes)
            if siguiente_estado not in visitados:
                accion = f"Entregar paquete en {ubicacion}"
                print(f"   ➕ Añadiendo a frontera -> Acción: {accion}")
                frontera.append((siguiente_estado, camino + [accion], costo_total))

        # Generar sucesores de movimiento vial
        for vecino in G1.successors(ubicacion):
            costo_arista = G1[ubicacion][vecino]['weight']
            siguiente_estado = (vecino, pendientes)
            if siguiente_estado not in visitados:
                accion = f"Moverse a {vecino}"
                print(f"   ➕ Añadiendo a frontera -> Acción: {accion} (Costo:
+{costo_arista} km)")
                frontera.append((siguiente_estado, camino + [accion], costo_total +
costo_arista))

        paso += 1

# Ejecutar simulación 1
simulacion_en_vivo_logistica(('Almacen', ('Cliente_A', 'Cliente_B')), ('Almacen', ()))



print("\n\n============================================================
=========")
print(" 2. INICIANDO SIMULACIÓN EN VIVO: ROBOT EN PLANTA INDUSTRIAL")
print("============================================================
=======")
```

```
time.sleep(2)

# Grafo de la cuadrícula del robot
G2 = nx.DiGraph()
obstaculos = {(1, 1), (2, 1)}
zonas_pesadas = {(1, 2): 3, (2, 2): 3}

for x in range(4):
    for y in range(4):
        if (x, y) in obstaculos: continue
        for dir_n, (nx_p, ny_p) in [('Norte', (x, y+1)), ('Sur', (x, y-1)), ('Este', (x+1, y)),
('Oeste', (x-1, y))]:
            if 0 <= nx_p < 4 and 0 <= ny_p < 4 and (nx_p, ny_p) not in obstaculos:
                costo = zonas_pesadas.get((nx_p, ny_p), 1)
                G2.add_edge((x, y), (nx_p, ny_p), weight=costo, direccion=dir_n)

def simulacion_en_vivo_robot(inicio, meta):
    frontera = deque([(inicio, [], 0)])
    visitados = set()
    paso = 1

    while frontera:
        actual, camino, energia = frontera.popleft()

        print(f"\n[Paso {paso}] Robot escaneando entorno...")
        print(f"  🤖 Posición de coordenadas (x, y) : {actual}")
        print(f"  🔋 Energía total consumida       : {energia} unidades")
        time.sleep(1.2) # Pausa visual de 1.2 segundos

        if actual == meta:
            print("\n" + "!"*55)
            print(f" ¡ÉXITO! Robot llegó a la Estación de Ensamblado (3,3).")
            print(f" Consumo energético total óptimo  : {energia} unidades")
            print(f" Secuencia de movimientos         : {' -> '.join(camino)}")
```

Gemini (Google DeepMind) / Modelo de Lenguaje de Gran Escala (LLM).

```
            print("!"*55)
            return

        if actual in visitados:
            print("   -> Celda ya procesada. Saltando...")
            continue
        visitados.add(actual)

        for vecino in G2.successors(actual):
            costo_movimiento = G2[actual][vecino]['weight']
            dir_mov = G2[actual][vecino]['direccion']
            if vecino not in visitados:
                accion = f"Mover {dir_mov} a {vecino}"
                print(f"   ➕ Casilla accesible encontrada -> {accion} (Costo:
{costo_movimiento})")
                frontera.append((vecino, camino + [accion], energia + costo_movimiento))

        paso += 1

# Ejecutar simulación 2
simulacion_en_vivo_robot((0, 0), (3, 3))
```

CONCLUSIONES

1. Eficiencia en la Representación: El modelado del espacio de estados influye drásticamente en la complejidad del problema. En el Problema 1, el tamaño del estado crece de forma exponencial $O(N \cdot 2^N)$ debido a la memoria de entregas pendientes, mientras que en el Problema 2 permanece polinomial $O(M \cdot N)$.
2. Utilidad de los Grafos Dirigidos y Ponderados: NetworkX facilita la abstracción de costos diferenciados (como distancias viales o consumo energético por zonas de tráfico). Esto permite aplicar posteriormente algoritmos informados como A^* utilizando heurísticas admisibles para encontrar soluciones globales óptimas.

Fuentes

Bohm, G. (s. f.). Introducción a los problemas de búsqueda y espacios de estados (Cap. 1-2).

NetworkX Developers. (s. f.). NetworkX: Software for complex networks (Versión 3.x) [Software de Python]. https://networkx.org/

Python Software Foundation. (s. f.). Python Language Reference (Versión 3.13) [Software de computadora]. https://www.python.org/

Russell, S., & Norvig, P. (2020). Inteligencia artificial: Un enfoque moderno (4ª ed.). Pearson Educación. (Referencia de apoyo teórico para agentes de búsqueda y grafos).

Gemini (Google DeepMind) / Modelo de Lenguaje de Gran Escala (LLM).